TASKFLOW

REST API with Authentication & Role-Based Access Control

Backend	Frontend
Node.js + Express.js	React 18 + Vite
PostgreSQL + Sequelize	Axios with JWT interceptors
JWT + bcrypt	React Context (Auth state)
Swagger (OpenAPI 3.0)	React Router v6

1. Project Overview

TaskFlow is a production-structured REST API built with Node.js and Express, backed by PostgreSQL via Sequelize ORM. It implements JWT-based authentication, role-based access control (user vs admin), and full CRUD operations for a task management entity. A React frontend connects to all endpoints and provides a functional dashboard UI.

What It Does

- Users can register, log in, and receive a JWT token used for all protected requests.
- Regular users can create, read, update, and delete their own tasks with ownership enforced server-side.
- Admin users get a separate panel to view and delete all users in the system.
- All API responses follow a consistent shape: { success, message, data, statusCode }.
- Interactive API documentation is available at /api/v1/docs via Swagger UI.

System Architecture

The project is split into two independent applications backend and frontend that communicate over HTTP.

Layer	Responsibility
React Frontend (Vite)	Auth pages, protected dashboard, CRUD task UI, toast notifications
Axios Interceptor	Auto-attaches Bearer token to every outgoing request
Express Routes /api/v1	Receives requests, applies middleware, delegates to controllers
Auth Middleware	Verifies JWT signature, attaches user object to req.user
Role Middleware	Checks req.user.role before allowing admin-only routes
Joi Validation	Validates and sanitizes all request body fields before they reach services
Services	All business logic DB queries, ownership checks, error throwing
PostgreSQL	Persistent storage users table, tasks table with FK relationship

2. API Reference

All endpoints are prefixed with /api/v1. The API uses standard HTTP methods and returns consistent JSON responses. Protected routes require a Bearer token in the Authorization header.

Authentication Endpoints

Method	Endpoint	Access	Description
POST	`/auth/register`	Public	Register a new user. Returns JWT on success.
POST	`/auth/login`	Public	Login with email + password. Returns JWT.
GET	`/auth/me`	User+	Returns the currently authenticated user's profile.

Task Endpoints

Method	Endpoint	Access	Description
GET	`/tasks`	User+	Fetch all tasks belonging to the current user.
POST	`/tasks`	User+	Create a new task with title, description, status, priority.
GET	`/tasks/:id`	User+	Get a specific task by ID (must be owner).
PUT	`/tasks/:id`	User+ (owner)	Update a task. Ownership enforced server-side.
DELETE	`/tasks/:id`	User+ (owner)	Delete a task. Ownership enforced server-side.

Admin Endpoints

Method	Endpoint	Access	Description
GET	`/admin/users`	Admin only	Retrieve all registered users in the system.
DELETE	`/admin/users/:id`	Admin only	Delete a user and cascade-delete their tasks.
GET	`/admin/tasks`	Admin only	Retrieve all tasks from all users.

Response Format

Every API response success or error follows this exact shape:

Success Response

```
{
  "success": true,
  "message": "Task created.",
  "statusCode": 201,
  "data": { ... }
}
```

Error Response

```
{
  "success": false,
  "message": "Invalid token.",
  "statusCode": 401,
  "data": null
}
```

3. Security Implementation

Password Hashing

All passwords are hashed using bcrypt with a salt round of 12 before storage. The raw password is never stored or logged. The User model's defaultScope permanently excludes the password_hash field from all database query results.

JWT Authentication

On login or registration, the server signs a JWT containing the user ID and role using a secret key. The token expires in 7 days. On the frontend, the token is stored in React state (AuthContext) never in localStorage or sessionStorage which means it cannot be accessed by JavaScript injection attacks and is automatically cleared when the tab closes.

Security Feature	Implementation
Password hashing	bcrypt, 12 salt rounds industry standard cost factor
JWT signing	HS256 algorithm, 7-day expiry, secret from environment variable
Token storage	React memory only never localStorage, sessionStorage, or cookies
Input validation	Joi schemas on every POST/PUT route rejects malformed payloads
Input sanitization	Joi strips unknown fields prevents mass assignment attacks
Ownership enforcement	Task service checks user_id match before any mutation
Rate limiting	20 requests / 15 min on /auth/* routes prevents brute force
Security headers	Helmet.js sets X-Frame-Options, CSP, HSTS, and 12 other headers
CORS policy	Strict origin whitelist only frontend URL accepted
Error messages	Generic auth errors never reveals if email exists or not

4. Scalability Design

Horizontal Scaling

JWT is stateless the server stores no session data. Multiple backend instances can run behind a load balancer (Nginx or AWS ALB) without shared session storage. Any instance independently validates any token using the shared JWT_SECRET.

Modular Structure to Microservices

Each feature module (auth, tasks, admin) is fully self-contained with its own routes, controller, service, and validator. Extracting any module into its own microservice requires no internal refactoring just move the folder and give it its own Express server and database connection.

Database Scaling

- Connection pooling: Sequelize maintains a pool of up to 10 connections, preventing exhaustion under concurrent load.
- Read replicas: Sequelize supports multiple read hosts natively write traffic goes to primary, reads spread across replicas.
- Indexing: email column is unique-indexed; user_id FK on tasks is indexed for fast ownership lookups.

API Versioning

All routes are prefixed /api/v1/. A new version (/api/v2/) can be added alongside v1, allowing non-breaking rollouts. Existing clients continue working while new clients migrate at their own pace.